Terminal ASCII Art That Types Itself

Turn any photo into an animated ASCII portrait & a neofetch-style GitHub profile — with just Python + Pillow.

No design tools · ~15 minutes · Works in GitHub READMEs

This guide shows the exact prompts and scripts used to build two things: (1) a simple ASCII portrait that *types itself out* like a terminal, and (2) a full **Andrew6rant-style** profile card (ASCII face on the left, a live-looking system-info panel on the right). Everything renders as a self-contained animated SVG that plays right inside a GitHub README.

> **Why an SVG?** GitHub blocks JavaScript in READMEs but *allows* SMIL — the animation syntax built into SVG. So an SVG can animate (typing, blinking cursor) with zero scripts, and it stays one portable file.

1 What you need

- **Python 3** (already on macOS/Linux; on Windows install from python.org)
- **Pillow** — the only dependency: `python3 -m pip install --user Pillow`
- An AI coding agent (**Claude Code**, Cursor, etc.) to generate/tweak the script
- A clear photo of your face (front-lit works best — more on this in Pro Tips)

2 The core idea (how it works)

Image → characters

Shrink the image to ~50–100 px wide, read each pixel's brightness, and map it to a character from a *ramp* that goes from dense (`@ $ B %`) to blank (`space`). Bright pixels become spaces, so light backgrounds wash out.

Characters → animated SVG

Write each row as SVG `<text>` . A clip-path that grows left→right reveals the row like typing; a `<rect>` that blinks is the cursor. `fill='freeze'` keeps the final frame.

3 The prompts (copy & paste)

Prompt A — the simple typing portrait

```
Make me a Python script (Pillow only) that turns any image into ASCII art. Create a
folder called 'ascii' in my Downloads — when I drop an image in and run the script
with no arguments, it auto-detects the image and saves an animated SVG next to it. The
SVG should type itself in line by line like a terminal — each row revealed left to
right with a block cursor sweeping across, then freeze when done. Use SMIL animation
so it even plays inside a GitHub README. Dark background, light monospace characters,
~100 characters wide. Bright areas should map to blank space so backgrounds wash out
clean.
```

Prompt B — the Andrew6rant-style profile

```
Now rebuild my GitHub profile README to look identical to github.com/Andrew6rant — a
single self-contained neofetch-style SVG: my ASCII self-portrait on the LEFT (fully
visible, not hidden), and a terminal info panel on the RIGHT (OS, Uptime, Host,
Kernel, IDE, Languages, Hobbies, Contact, GitHub Stats) with orange keys, blue values
and dotted leaders. Generate dark_mode.svg + light_mode.svg and reference them with a
<picture> tag. Add a blinking green terminal cursor as the only animation. Crop the
photo tight to my face, lift the shadows and use enough resolution so the FACE reads
clearly. Then replace my README and push it.
```

> **Pro tip:** paste Prompt A first, get it working on any image, *then* paste Prompt B. Iterating in two steps gives the agent a working base to build the fancier version on.

4 Step-by-step

1. Open your AI coding agent in a terminal.
2. Paste **Prompt A**. It creates `~/Downloads/ascii/` and the script.
3. Drop a photo into that folder and run `python3 ascii_terminal.py`.
4. Open the generated `.svg` in a browser to watch it type itself.
5. Paste **Prompt B** to generate the neofetch profile (`dark_mode.svg` + `light_mode.svg`).
6. Put it on GitHub: create a repo named exactly like your username (`you/you`), add the SVGs, and use this README:

```
<a href="https://github.com/YOURNAME">
  <picture>
    <source media="(prefers-color-scheme: dark)"
            srcset="https://raw.githubusercontent.com/YOURNAME/YOURNAME/main/dark_mode.svg">
    <img alt="YOURNAME"
src="https://raw.githubusercontent.com/YOURNAME/YOURNAME/main/light_mode.svg">
  </picture>
</a>
```

5 Pro tips for a clear face

- **Crop tight to the head.** The face must fill the grid — a full-body shot turns into mush. Crop from just above the hair to the shoulders.
- **Backlit / dark photo? Lift the shadows.** Apply a gamma curve (`v ** 0.55`) before contrast so features separate.

- **Resolution vs. layout.** 43 columns is compact but soft; 56–70 columns reads far clearer. Render extra rows with a smaller font so it still fits.
- **Bright → blank.** Map the brightest pixels to a space so busy backgrounds disappear and only you remain.
- **Remove the background first** (any tool) for the cleanest portrait.

6 Script A — `ascii_terminal.py` (simple typing portrait)

Drop any image in the folder, run with no arguments, get an animated SVG next to it.

```
#!/usr/bin/env python3
"""
ascii_terminal.py — Turn any image into ASCII art that types itself out
like a terminal, saved as a self-contained animated SVG.

Usage:
    Drop an image into this folder and run with no arguments:
        python3 ascii_terminal.py

    It auto-detects the newest image in the folder, converts it to ASCII,
    and writes an animated .svg next to it. The SVG uses SMIL animation so
    it plays in a browser AND inside a GitHub README.

    Optional: pass an image path explicitly:
        python3 ascii_terminal.py path/to/photo.png

Dependencies: Pillow only.
"""

import sys
import os
import glob
from PIL import Image

# ---------------------------------------------------------------------------
# Tunables
# ---------------------------------------------------------------------------
WIDTH = 100                 # target width in characters (~100 wide)
CHAR_ASPECT = 0.52          # monospace glyphs are ~2x taller than wide
CONTRAST = 1.15             # >1 boosts contrast, <1 flattens it

# Bright -> blank, dark -> dense. This makes bright backgrounds wash out.
# First char is a plain space so highlights become clean empty space.
RAMP = " .'`^\",:;Il!i~+_-?][}{1)(|/tfjrxnuvczXYUJCLQ0OZmwqpdbkhao*#MW&8%B@$"

# Terminal look
BG_COLOR = "#0d1117"        # dark background (GitHub dark)
FG_COLOR = "#c9d1d9"        # light monospace text
CURSOR_COLOR = "#3fb950"    # green terminal cursor
FONT_SIZE = 12              # px
LINE_HEIGHT = 13            # px per row
CHAR_WIDTH = 7.2            # px per glyph (monospace advance)
PAD = 16                    # px padding around the art

ROW_DURATION = 0.28         # seconds to type one row
PIXELS_LOOK_HOLD = 1.2      # seconds frozen at the end before it settles

IMAGE_EXTS = ("*.jpg", "*.jpeg", "*.png", "*.gif", "*.bmp", "*.webp",
              "*.tif", "*.tiff", "*.JPG", "*.JPEG", "*.PNG")

# ---------------------------------------------------------------------------
# Careers with Aniket · Build in public, get hired ____ anik8.me · github.com/aniketsingh1023
# Image -> ASCII
```

```
# ---------------------------------------------------------------------------
def image_to_ascii(path, width=WIDTH):
    from PIL import ImageOps, ImageEnhance
    img = Image.open(path).convert("L")  # grayscale luminance

    # Spread the tonal range so faces don't turn to mush and bright
    # backgrounds push all the way to white (-> blank space).
    img = ImageOps.autocontrast(img, cutoff=1)

    if CONTRAST != 1.0:
        img = ImageEnhance.Contrast(img).enhance(CONTRAST)

    w, h = img.size
    new_w = width
    new_h = max(1, int(width * (h / w) * CHAR_ASPECT))
    img = img.resize((new_w, new_h))

    px = img.load()
    n = len(RAMP) - 1
    rows = []
    for y in range(new_h):
        chars = []
        for x in range(new_w):
            lum = px[x, y]                 # 0 = black, 255 = white
            # Bright areas -> start of ramp (space); dark -> dense glyphs
            idx = int((255 - lum) / 255 * n)
            chars.append(RAMP[idx])
        rows.append("".join(chars).rstrip())
    return rows


# ---------------------------------------------------------------------------
# ASCII -> animated SVG (SMIL, GitHub-README friendly)
# ---------------------------------------------------------------------------
def _xml_escape(s):
    return (s.replace("&", "&amp;").replace("<", "&lt;")
             .replace(">", "&gt;").replace('"', "&quot;"))


def ascii_to_svg(rows, out_path):
    n_rows = len(rows)
    max_cols = max((len(r) for r in rows), default=1)

    art_w = max_cols * CHAR_WIDTH
    art_h = n_rows * LINE_HEIGHT
    svg_w = int(art_w + PAD * 2)
    svg_h = int(art_h + PAD * 2)

    total = n_rows * ROW_DURATION + PIXELS_LOOK_HOLD  # full loop length

    parts = []
    parts.append(
        f'<svg xmlns="http://www.w3.org/2000/svg" width="{svg_w}" '
        f'height="{svg_h}" viewBox="0 0 {svg_w} {svg_h}" '
        f'font-family="monospace">'
    )
    parts.append(f'<rect width="100%" height="100%" fill="{BG_COLOR}"/>')
    parts.append(
        f'<style>text{{font-family:Menlo,Consolas,"DejaVu Sans Mono",'
        f'monospace;font-size:{FONT_SIZE}px;white-space:pre;'
        f'dominant-baseline:hanging;}}</style>'
    )

    for i, row in enumerate(rows):
        y = PAD + i * LINE_HEIGHT
        start = i * ROW_DURATION
        row_len = max(len(row), 1)
        row_px = row_len * CHAR_WIDTH
```

```
        # Clip rectangle that sweeps left->right to reveal the row.
        clip_id = f"c{i}"
        parts.append(f'<clipPath id="{clip_id}">')
        parts.append(
            f'<rect x="{PAD}" y="{y}" width="0" height="{LINE_HEIGHT}">'
            f'<animate attributeName="width" from="0" to="{row_px:.1f}" '
            f'begin="{start:.3f}s" dur="{ROW_DURATION:.3f}s" '
            f'fill="freeze" calcMode="linear"/>'
            f'</rect>'
        )
        parts.append('</clipPath>')

        # The row text, revealed through its clip.
        parts.append(
            f'<text x="{PAD}" y="{y}" fill="{FG_COLOR}" '
            f'clip-path="url(#{clip_id})" '
            f'xml:space="preserve">{_xml_escape(row)}</text>'
        )

        # Block cursor that sweeps across this row while it types,
        # then vanishes when the row is done.
        parts.append(
            f'<rect y="{y}" width="{CHAR_WIDTH:.1f}" '
            f'height="{FONT_SIZE}" fill="{CURSOR_COLOR}" opacity="0">'
            f'<animate attributeName="x" from="{PAD}" '
            f'to="{PAD + row_px:.1f}" begin="{start:.3f}s" '
            f'dur="{ROW_DURATION:.3f}s" fill="freeze"/>'
            f'<set attributeName="opacity" to="0.85" begin="{start:.3f}s"/>'
            f'<set attributeName="opacity" to="0" '
            f'begin="{start + ROW_DURATION:.3f}s"/>'
            f'</rect>'
        )

    parts.append('</svg>')

    with open(out_path, "w", encoding="utf-8") as f:
        f.write("\n".join(parts))
    return total


# ---------------------------------------------------------------------------
# Auto-detect the image to convert
# ---------------------------------------------------------------------------
def find_image(folder):
    candidates = []
    for pat in IMAGE_EXTS:
        candidates.extend(glob.glob(os.path.join(folder, pat)))
    # Ignore anything we might have generated
    candidates = [c for c in set(candidates) if not c.lower().endswith(".svg")]
    if not candidates:
        return None
    # Newest file wins (the one you just dropped in)
    return max(candidates, key=os.path.getmtime)


def main():
    folder = os.path.dirname(os.path.abspath(__file__))

    if len(sys.argv) > 1:
        img_path = os.path.abspath(sys.argv[1])
        if not os.path.exists(img_path):
            print(f"! Not found: {img_path}")
            sys.exit(1)
    else:
        img_path = find_image(folder)
        if not img_path:
            print("! No image found. Drop a .jpg/.png into this folder and "
```

```
                "run again.")
            sys.exit(1)

    print(f"→ Reading {os.path.basename(img_path)}")
    rows = image_to_ascii(img_path)

    out_path = os.path.splitext(img_path)[0] + ".svg"
    duration = ascii_to_svg(rows, out_path)

    print(f"✓ Wrote {os.path.basename(out_path)} "
          f"({len(rows)} rows, ~{duration:.1f}s animation)")
    print(f"  Open it in a browser:  open \"{out_path}\"")


if __name__ == "__main__":
    main()
```

7 Script B — `andrew_style.py` (neofetch profile)

Reads `aniket_portrait.png`, writes `dark_mode.svg` and `light_mode.svg`. Edit the `INFO` list with your own details.

```
#!/usr/bin/env python3
"""
Build an Andrew6rant-style neofetch profile SVG:
  - ASCII self-portrait on the left (fully visible, static)
  - terminal info panel on the right that types itself in line by line
Generates dark_mode.svg and light_mode.svg (self-contained, SMIL).
"""
from PIL import Image, ImageOps, ImageEnhance

# ---- 1. ASCII portrait ------------------------------------------------
# Higher resolution (56 wide) so the FACE reads clearly, rendered with a
# small font / tight line-height so it still fits Andrew's ~530px height.
ASCII_COLS = 56
ASCII_FS = 13     # portrait font-size (px)
ASCII_LH = 14     # portrait line-height (px)
ASCII_CW = 7.8    # portrait char advance (px) at that font size
RAMP = " .'`^\",:;Il!i~+_-?][}{1)(|/tfjrxnuvczXYUJCLQ0OZmwqpdbkhao*#MW&8%B@$"


def portrait_rows(path, cols=ASCII_COLS):
    """Backlit-face friendly: lift shadows (gamma), autocontrast, boost."""
    im = Image.open(path).convert("L")
    im = im.point(lambda v: int(((v / 255.0) ** 0.55) * 255))  # lift shadows
    im = ImageOps.autocontrast(im, cutoff=2)
    im = ImageEnhance.Contrast(im).enhance(1.25)
    w, h = im.size
    # keep aspect undistorted for a cell of ASCII_CW x ASCII_LH
    rows = max(1, int(cols * (h / w) * (ASCII_CW / ASCII_LH)))
    im = im.resize((cols, rows))
    px = im.load()
    n = len(RAMP) - 1
    out = []
    for y in range(rows):
        out.append("".join(RAMP[int((255 - px[x, y]) / 255 * n)]
                           for x in range(cols)).rstrip())
    return out


ASCII_ROWS = portrait_rows("aniket_portrait.png")

# ---- 2. Right-panel content -------------------------------------------
# Each info row: (dot-key segments, value). Key segments render orange,
```

```
# joined by "." separators; dotted leader aligns the value column.
NAME = "aniket@singh"

INFO = [
    ("header", NAME),
    ("kv", (["OS"], "macOS · Linux · Android")),
    ("kv", (["Uptime"], "21 years")),
    ("kv", (["Host"], "Maxcode IT Solutions")),
    ("kv", (["Kernel"], "Chief Technology Officer (CTO)")),
    ("kv", (["IDE"], "VSCode, Cursor, Claude Code")),
    ("blank", None),
    ("kv", (["Languages", "Programming"], "JavaScript, TypeScript, Python")),
    ("kv", (["Languages", "Computer"], "HTML, CSS, JSON, YAML, LaTeX")),
    ("kv", (["Languages", "Real"], "English, Hindi")),
    ("blank", None),
    ("kv", (["Hobbies", "Software"], "Hackathons, Open Source, AI")),
    ("kv", (["Hobbies", "Building"], "Startups, Dev Tools")),
    ("blank", None),
    ("section", "Contact"),
    ("kv", (["Email", "Personal"], "aniketsinghn10@gmail.com")),
    ("kv", (["Portfolio"], "anik8.me")),
    ("kv", (["LinkedIn"], "aniketsingh1023")),
    ("kv", (["GitHub"], "aniketsingh1023")),
    ("blank", None),
    ("section", "GitHub Stats"),
    ("stats1", None),
    ("stats2", None),
    ("stats3", None),
]

VALUE_COL = 26   # column where value text begins (monospace chars)

THEMES = {
    "dark": dict(bg="#161b22", text="#c9d1d9", key="#ffa657",
                 value="#a5d6ff", cc="#616e7f", add="#3fb950", dele="#f85149"),
    "light": dict(bg="#ffffff", text="#24292f", key="#953800",
                  value="#0a3069", cc="#6e7781", add="#1a7f37", dele="#cf222e"),
}


def esc(s):
    return (s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;"))


def leader(prefix_len):
    """Dots needed to reach the value column."""
    return max(1, VALUE_COL - prefix_len)


def kv_line(keys, value):
    """Return tspans for a '. Key.Sub: ..... value' line."""
    key_txt = ".".join(keys)
    # prefix chars counted: ". " + key + ":"
    prefix_len = 2 + len(key_txt) + 1
    dots = leader(prefix_len)
    key_spans = ('<tspan class="key">'
                 + '</tspan>.<tspan class="key">'.join(esc(k) for k in keys)
                 + '</tspan>')
    return (f'<tspan class="cc">. </tspan>{key_spans}'
            f'<tspan class="cc">:</tspan>'
            f'<tspan class="cc"> {"." * dots} </tspan>'
            f'<tspan class="value">{esc(value)}</tspan>')


CW = 10.0       # px per monospace char (info panel, wide-font safe)
INFO_X = 500    # info panel left edge (clears the ASCII portrait)
W, H = 1120, 540
```

```
def build_svg(theme_name):
    t = THEMES[theme_name]
    parts = []
    parts.append(
        f"<svg xmlns='http://www.w3.org/2000/svg' "
        f"font-family=\"Consolas,'DejaVu Sans Mono',monospace\" "
        f"width='{W}px' height='{H}px' font-size='16px'>")
    parts.append(
        "<style>"
        f".key{{fill:{t['key']};}} .value{{fill:{t['value']};}} "
        f".cc{{fill:{t['cc']};}} .add{{fill:{t['add']};}} "
        f".del{{fill:{t['dele']};}} "
        "text,tspan{white-space:pre;} "
        "/* type-in reveal for each info line */"
        "</style>")
    parts.append(f"<rect width='{W}px' height='{H}px' fill='{t['bg']}' rx='15'/>")

    # ----- ASCII portrait: fully visible, static (small dense font) -----
    parts.append(f"<text x='15' y='24' fill='{t['text']}' "
                 f"font-size='{ASCII_FS}px'>")
    y = 24
    for row in ASCII_ROWS:
        parts.append(f"<tspan x='15' y='{y}'>{esc(row)}</tspan>")
        y += ASCII_LH
    parts.append("</text>")

    # ----- Right info panel: fully visible (the whole image is just there) --
    px = INFO_X
    y = 30
    # dashes fill the header/section rules out to the right edge
    n_dash = int((W - px) / CW) - 16
    for i, (kind, payload) in enumerate(INFO):
        if kind == "header":
            dash = "-" * max(4, n_dash - len(payload))
            body = (f"<tspan x='{px}' y='{y}' fill='{t['text']}'>{esc(payload)}"
                    f"</tspan><tspan class='cc'> -{dash}-</tspan>")
        elif kind == "section":
            dash = "-" * max(4, n_dash - len(payload) - 2)
            body = (f"<tspan x='{px}' y='{y}' fill='{t['text']}'>- {esc(payload)}"
                    f"</tspan><tspan class='cc'> -{dash}-</tspan>")
        elif kind == "blank":
            body = f"<tspan x='{px}' y='{y}' class='cc'>. </tspan>"
        elif kind == "kv":
            keys, value = payload
            body = f"<tspan x='{px}' y='{y}'>{kv_line(keys, value)}</tspan>"
        elif kind == "stats1":
            body = (f"<tspan x='{px}' y='{y}'><tspan class='cc'>. </tspan>"
                    f"<tspan class='key'>Repos</tspan>"
                    f"<tspan class='cc'> ..... </tspan>"
                    f"<tspan class='value'>72</tspan>"
                    f"<tspan class='cc'>  |  </tspan>"
                    f"<tspan class='key'>Stars</tspan>"
                    f"<tspan class='cc'> ..... </tspan>"
                    f"<tspan class='value'>23</tspan></tspan>")
        elif kind == "stats2":
            body = (f"<tspan x='{px}' y='{y}'><tspan class='cc'>. </tspan>"
                    f"<tspan class='key'>Commits</tspan>"
                    f"<tspan class='cc'> ... </tspan>"
                    f"<tspan class='value'>2,936</tspan>"
                    f"<tspan class='cc'>  |  </tspan>"
                    f"<tspan class='key'>Followers</tspan>"
                    f"<tspan class='cc'> . </tspan>"
                    f"<tspan class='value'>49</tspan></tspan>")
        elif kind == "stats3":
            body = (f"<tspan x='{px}' y='{y}'><tspan class='cc'>. </tspan>"
                    f"<tspan class='key'>Member Since</tspan>"
                    f"<tspan class='cc'> ... </tspan>"
```

```
                    f"<tspan class='value'>March 2023</tspan>"
                    f"<tspan class='cc'>  |  </tspan>"
                    f"<tspan class='key'>Location</tspan>"
                    f"<tspan class='cc'> . </tspan>"
                    f"<tspan class='value'>Indore, IN</tspan></tspan>")

        parts.append(f"<text>{body}</text>")
        y += 20

    # ----- blinking terminal prompt + cursor (the "alive" animation) -----
    prompt_y = y + 8
    parts.append(
        f"<text x='{px}' y='{prompt_y}' fill='{t['add']}'>aniket@github</text>"
        f"<text x='{px + 13 * int(CW)}' y='{prompt_y}' fill='{t['text']}'>:~$</text>")
    cur_x = px + 17 * int(CW)
    parts.append(
        f"<rect x='{cur_x}' y='{prompt_y - 13}' width='{int(CW)}' height='17' "
        f"fill='{t['add']}'>"
        f"<animate attributeName='opacity' values='1;1;0;0' dur='1.1s' "
        f"keyTimes='0;0.5;0.5;1' repeatCount='indefinite'/></rect>")

    parts.append("</svg>")
    return "\n".join(parts)


for name in ("dark", "light"):
    out = f"{name}_mode.svg"
    with open(out, "w", encoding="utf-8") as f:
        f.write(build_svg(name))
    print("wrote", out)
```

> **Made something with this?** Tag **Careers with Aniket** — I feature the best profiles. Learn to build things that get you noticed at anik8.me.